✍️ OOP Handwritten Notes (सरल हिंदी नोट्स)

Concept: Object Oriented Programming & Live Screen Code Breakdown

📌 OOP (Object-Oriented Programming) क्या है?
OOP कंप्यूटर प्रोग्रामिंग का वह तरीका है जहाँ कोड को असली दुनिया (Real Life) की तरह ढाला जाता है। इसमें डेटा और फंक्शंस को एक साथ **"Objects"** (वस्तुओं) में बांधा जाता है, जिससे बड़े प्रोजेक्ट्स को समझना, संभालना और सुरक्षित रखना बहुत आसान हो जाता है।

1. दो मुख्य नींव: Class और Object

📝 1. Class (नक्शा / Blueprint)

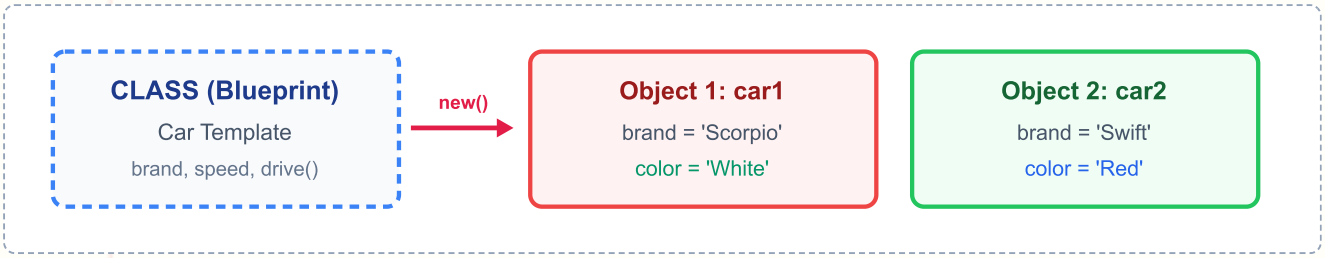
यह केवल एक कागजी ढांचा या फॉर्म है। इसमें बताया जाता है कि आने वाले ऑब्जेक्ट के पास क्या-क्या गुण (Variables) और क्या-क्या काम (Functions) होंगे। यह मेमोरी में जगह नहीं लेता।

उदा: बैंक फॉर्म या घर का नक्शा।

🧱 2. Object (असली वस्तु / Instance)

Class रूपी नक्शे से जब असली रूप तैयार होता है, उसे Object कहते हैं। यह कंप्यूटर की RAM मेमोरी में जगह लेता है और इसका अपना अलग डेटा होता है।

उदा: भरा हुआ फॉर्म या बना हुआ मकान।



2. OOP के 4 मुख्य खंभे (Pillars)

🔐 1. Encapsulation

डेटा (variables) और काम (methods) को एक बंद कैप्सूल में सुरक्षित रखना ताकि बाहर से कोई गलती से डेटा न बिगाड़ सके।

🎭 2. Polymorphism

एक नाम लेकिन परिस्थिति के अनुसार अलग-अलग काम (Poly = अनेक, Morph = रूप)। एक ही फंक्शन अलग बिहेव करे।

👨‍👦 3. Inheritance

पुरानी क्लास (Parent) के गुणों को नई क्लास (Child) में सीधे पाना। इससे कोड दोबारा लिखने की ज़रूरत नहीं पड़ती।

🕹️ 4. Abstraction

अंदर की जटिल कोडिंग छुपाकर यूज़र को केवल काम का बटन देना। जैसे कार का एक्सीलेटर दबाना बिना इंजन समझे।

💡 Python में `self` का क्या मतलब है?
जब हम क्लास के अंदर कोई फंक्शन लिखते हैं, तो पहला पैरामीटर `self` होता है। `self` का मतलब है *"वर्तमान ऑब्जेक्ट स्वयं"*। यह बताता है कि यह डेटा किस ऑब्जेक्ट (p1 या a) से ताल्लुक रखता है।

🖥️ आपकी स्क्रीन वाले कोड का सम्पूर्ण विश्लेषण

File 1: test_person.py | File 2: test_account.py

Page 02 • Live Screen Code

📸 कोड 1: Person क्लास (Screen 3 - `test_person.py`)

```
class Person: def __init__(self): self.firt_name = '' self.last_name = '' def get_details(self): print('person information') print('first_name:', self.firt_name) print('last_name:', self.last_name) p = Person() p.firt_name = 'prerna'; p.last_name = 'thakur' p1 = Person() p1.firt_name = 'rakesh'; p1.last_name = 'panchal' p1.get_details()
```

📝 **समझना क्या है:** यहाँ `Person` एक ढांचा है। जब `p = Person()` लिखा तो *Prerna* का ऑब्जेक्ट बना, और `p1 = Person()` से *Rakesh* का ऑब्जेक्ट बना। जब `p1.get_details()` चला तो केवल राकेश का नाम प्रिंट हुआ क्योंकि `self` ने `p1` के डेटा को पहचाना।

📸 कोड 2: Account क्लास (Screen 1 & 2 - `test_account.py`)

```
class Account: def __init__(self): self.bank_name = ''; self.account_number = ''; self.balance = 0 def deposite(self, amount): self.balance += amount print(amount, 'deposited, updated balance:', self.balance) def withdrawl(self, amount): self.balance -= amount print(amount, 'withdrawl, updated balance:', self.balance) a = Account() a.balance = 1000 a.deposite(4000) # बैलेंस हुआ: 5000 a.withdrawl(500) # बैलेंस बचा: 4500
```

🎯 आपकी स्क्रीन कोड की शब्दावली (Quick Cheatsheet):

कोड शब्द	सरल भाषा में अर्थ	स्क्रीन पर इसका रोल
`def __init__(self)`	Constructor (शुरुआती फंक्शन)	नया खाता या व्यक्ति बनते ही वेरिएबल्स को खाली/डिफ़ॉल्ट मान देना।
`self.balance += amount`	Encapsulated Action	अकाउंट के अंदर सुरक्षित रूप से पैसे जोड़ना (डिपॉजिट लॉजिक)।
`p = Person()`	Instantiating (ऑब्जेक्ट बनाना)	क्लास रूपी नक्शे से असली अलग-अलग मेमोरी ब्लॉक तैयार करना।

⚠️ **क्लास में स्पेलिंग सुधार टिप:** कोड में `firt_name` की जगह `first_name`, `deposite` की जगह `deposit` और `withdrawl` की जगह `withdraw` लिखना स्टैंडर्ड पायथन कन्वेंशन है।